7

Features – How Algorithms See the World

So far in this book, we suggested a number of ways and a number of reasons for creating, extracting, or, otherwise, manipulating features. In this chapter, we will address this topic head on. The right features, sometimes called **attributes**, are the central component for machine learning models. A sophisticated model with the wrong features is worthless. Features are how our applications see the world. For all but the most simple tasks, we will process our features before feeding them to a model. There are many interesting ways in which we can do this, and it is such an important topic that it's appropriate to devote an entire chapter to it.

It has only been in the last decade or so that machine learning models have been routinely using tens of thousands of features or more. This allows us to tackle many different problems, such as those where our feature set is large compared to the number of samples. Two typical applications are genetic analysis and text categorization. For genetic analysis, our variables are a set of **gene expression coefficients**. These are based on the number of **mRNA** present in a sample, for example, taken from a tissue biopsy. A classification task might be performed to predict whether a patient has cancer or not. The number of training and test samples together may be a number less than 100. On the other hand, the number of variables in the raw data may range from 6,000 to 60,000. Not only will this translate to a large number of features, it also means that the range of values between features is quite large too. In this chapter, we will cover the following topics:

- Feature types
- Operations and statistics
- Structured features
- Transforming features
- Principle component analysis

2016. Packt Publishing.
All rights reserved. May not be reproduced in any form without permission from the publisher, except fair uses permitted under U.S. or applicable copyright law.

EBSCO Publishing: eBook Collection (EBSCOhost) printed on 3/19/2026 5:26:39 PM UTC via TORRENS GLOBAL EDUCATION SERVICES PTY LTD 1218065; David Julian; Designing Machine Learning Systems with Python : Key Design Strategies to Create Intelligent Systems Account:ehost.

Feature types

There are three distinct types of features: quantitative, ordinal, and categorical. We can also consider a fourth type of feature—the Boolean—as this type does have a few distinct qualities, although it is actually a type of categorical feature. These feature types can be ordered in terms of how much information they convey. Quantitative features have the highest information capacity followed by ordinal, categorical, and Boolean.

Let's take a look at the tabular analysis:

Feature type	Order	Scale	Tendency	Dispersion	Shape
Quantitative	Yes	Yes	Mean	Range, variance, and standard deviation	Skewness, kurtosis
Ordinal	Yes	No	Median	Quantiles	NA
Categorical	No	No	Mode	NA	NA

The preceding table shows the three types of features, their statistics, and properties. Each feature inherits the statistics from the features from the next row it in the table. For example, the measurement of central tendency for quantitative features includes the median and mode.

Quantitative features

The distinguishing characteristic of quantitative features is that they are continuous, and they usually involve mapping them to real numbers. Often, feature values can be mapped to a subset of real numbers, for example, expressing age in years; however, care must be taken to use the full scale when calculating statistics, such as mean or standard deviation. Because quantitative features have a meaningful numeric scale, they are often used in geometric models. When they are used in tree models, they result in a binary split, for example, using a threshold value where values above the threshold go to one child and values equal to or below the threshold go to the other child. Tree models are insensitive to monotonic transformations of scale, that is, transformations that do not change the ordering of the feature values. For example, it does not matter to a tree model if we measure length in centimeters or inches, or use a logarithmic or linear scale, we simply have to change the threshold values to the same scale. Tree models ignore the scale of quantitative features and treat them as ordinal. This is also true for rule-based models. For probabilistic models, such as the naïve Bayes classifier, quantitative features need to be discretized into a finite number of bins, and therefore, converted to categorical features.

EBSCOhost: eBook Collection (EBSCOhost) printed on 3/19/2026 5:26:39 PM UTC via TORRENS GLOBAL EDUCATION SERVICES PTY LTD. All use subject to https://www.ebsco.com/terms-of-use.

Ordinal features

Ordinal features are features that have a distinct order but do not have a scale. They can be encoded as integer values; however, doing so does not imply any scale. A typical example is that of house numbers. Here, we can discern the position of a house on a street by its number. We assume that house number 1 will come before house number 20 and that houses with the numbers 10 and 11 would be located close to each other. However, the size of the number does not imply any scale; for example, there is no reason to believe that house number 20 will be larger than house number 1. The domain of an ordinal feature is a totally ordered set such as a set of characters or strings. Because ordinal features lack a linear scale, it does not make sense to add or subtract them; therefore, operations such as averaging ordinal features do not usually make sense or yield any information about the features. Similar to quantitative features in tree models, ordinal features result in a binary split. In general, ordinal features are not readily used in most geometric models. For example, linear models assume a Euclidian instance space where feature values are treated as Cartesian coordinates. For distance-based models, we can use ordinal features if we encode them as integers and the distance between them is simply their difference. This is sometimes referred to as the **hamming distance**.

Categorical features

Categorical features, sometimes called **nominal features**, do not have any ordering or scale, and therefore, they do not allow any statistical summary apart from the mode indicating the most frequent occurrence of a value. Categorical features are often best handled by probabilistic models; however, they can also be used in distance-based models using the hamming distance and by setting the distance to 0 for equal values and 1 for unequal values. A subtype of categorical features is the **Boolean feature,** which maps into the Boolean values of true or false.

Operations and statistics

Features can be defined by the allowable operations that can be performed on them. Consider two features: a person's age and their phone number. Although both these features can be described by integers, they actually represent two very different types of information. This is clear when we see which operations we can usefully perform on them. For example, calculating the average age of a group of people will give us a meaningful result; calculating the average phone number will not.

We can call the range of possible calculations that can be performed on a feature as its statistics. These statistics describe three separate aspects of data. These are—its **central tendency**, its **dispersion**, and its **shape**.

EBSCOhost: eBook Collection (EBSCOhost) printed on 3/19/2026 5:26:39 PM UTC via TORRENS GLOBAL EDUCATION SERVICES PTY LTD. All use subject to https://www.ebsco.com/terms-of-use.

To calculate the central tendency of data, we usually use one or more of the following statistics: the mean (or average), the median (or the middle value in an ordered list), and the mode (or the majority of all values). The mode is the only statistic that can be applied to all data types. To calculate the median, we need feature values that can be somehow ordered, that is ordinal or quantitative. To calculate the mean, values must be expressed on some scale, such as the linear scale. In other words they will need to be quantitative features.

The most common way of calculating dispersion is through the statistics of variance or standard deviation. These are both really the same measure but on different scales, with standard deviation being useful because it is expressed on the same scale as the feature itself. Also, remember that the absolute difference between the mean and the median is never larger than the standard deviation. A simpler statistic for measuring dispersion is the range, which is just the difference between the minimum and maximum values. From here, of course, we can estimate the feature's central tendency by calculating the mid-range point. Another way to measure dispersion is using units such as percentiles or deciles to measure the ratio of instances falling below a particular value. For example, the pth percentile is the value that p percent of instances fall below.

Measuring shape statistics is a little more complicated and can be understood using the idea of the **central moment** of a sample. This is defined as follows:

$$m_k = \frac{1}{n}\sum_{i=1}^{n}\left(x_i - \mu\right)^k$$

Here, n is the number of samples, μ is the sample mean, and k is an integer. When $k = 1$, the first central moment is 0 because this is simply the average deviation from the mean, which is always 0. The second central moment is the average squared deviation from the mean, which is the variance. We can define **skewness** as follows:

$$\frac{m_3}{\sigma^3}$$

Here σ is the standard deviation. If this formula gives a value that is positive, then there are more instances with values above the mean rather than below. The data, when graphed, is skewed to the right. When the skew is negative, the converse is true.

EBSCOhost: eBook Collection (EBSCOhost) printed on 3/19/2026 5:26:39 PM UTC via TORRENS GLOBAL EDUCATION SERVICES PTY LTD. All use subject to https://www.ebsco.com/terms-of-use.

We can define **kurtosis** as a similar relationship for the fourth central moment:

$$\frac{m_4}{\sigma^4}$$

It can be shown that a normal distribution has a kurtosis of 3. At values above this, the distribution will be more *peaked*. At kurtosis values below 3, the distribution will be *flatter*.

We previously discussed the three types of data, that is, categorical, ordinal, and quantitative.

Machine learning models will treat the different data types in very distinct ways. For example, a decision tree split on a categorical feature will give rise to as many children as there are values. For ordinal and quantitative features, the splits will be binary, with each parent giving rise to two children based on a threshold value. As a consequence, tree models treat quantitative features as ordinal, ignoring the features scale. When we consider probabilistic models such as the **Bayes classifier**, we can see that it actually treats ordinal features as categorical, and the only way in which it can handle quantitative features is to turn them into a finite number of discrete values, therefore converting them to categorical data.

Geometric models, in general, require features that are quantitative. For example, linear models operate in a Euclidean instance space, with the features acting as Cartesian coordinates. Each feature value is considered as a scalar relationship to other feature values. Distance-based models, such as the k-nearest neighbor, can incorporate categorical features by setting the distance to 0 for equal values and 1 for unequal values. Similarly, we can incorporate ordinal features into distance-based models by counting the number of values between two values. If we are encoding feature values as integers, then the distance is simply the numerical difference. By choosing an appropriate distance metric, it is possible to incorporate ordinal and categorical features into distance-based models.

EBSCOhost: eBook Collection (EBSCOhost) printed on 3/19/2026 5:26:39 PM UTC via TORRENS GLOBAL EDUCATION SERVICES PTY LTD. All use subject to https://www.ebsco.com/terms-of-use.

Structured features

We assume that each instance can be represented as a vector of feature values and that all relevant aspects are represented by this vector. This is sometimes called an **abstraction** because we filter out unnecessary information and represent a real-world phenomena with a vector. For example, representing the entire works of Leo Tolstoy as a vector of word frequencies is an abstraction. We make no pretense that this abstraction will serve any more than a very particular limited application. We may learn something about Tolstoy's use of language and perhaps elicit some information regarding the sentiment and subject of Tolstoy's writing. However, we are unlikely to gain any significant insights into the broad canvas of the 19th century Russia portrayed in these works. A human reader, or a more sophisticated algorithm, will gain these insights not from the counting of each word but by the structure that these words are part of.

We can think of structured features in a similar way to how we may think about queries in a database programming language, such as SQL. A SQL query can represent an aggregation over variables to do things such as finding a particular phrase or finding all the passages involving a particular character. What we are doing in a machine learning context is creating another feature with these aggregate properties.

Structured features can be created prior to building the model or as part of the model itself. In the first case, the process can be understood as being a translation from the first order logic to a propositional logic. A problem with this approach is that it can create an explosion in the number of potential features as a result of combinations with existing features. Another important point is that, in the same way that in SQL one clause can cover a subset of another clause, structural features can also be logically related. This is exploited in the branch of machine learning that is particularly well suited to natural language processing, known as **inductive logic programming**.

Transforming features

When we transform features, our aim, obviously, is to make them more useful to our models. This can be done by adding, removing, or changing information represented by the feature. A common feature transformation is that of changing the feature type. A typical example is **binarization**, that is, transforming a categorical feature into a set of binary ones. Another example is changing an ordinal feature into a categorical feature. In both these cases, we lose information. In the first instance, the value of a single categorical feature is mutually exclusive, and this is not conveyed by the binary representation. In the second instance, we lose the ordering information. These types of transformations can be considered inductive because they consist of a well-defined logical procedure that does not involve an objective choice apart from the decision to carry out these transformations in the first place.

EBSCOhost: eBook Collection (EBSCOhost) printed on 3/19/2026 5:26:39 PM UTC via TORRENS GLOBAL EDUCATION SERVICES PTY LTD. All use subject to https://www.ebsco.com/terms-of-use.

Binarization can be easily carried out using the `sklearn.preprocessing.Binarizer` module. Let's take a look at the following commands:

```
from sklearn.preprocessing import Binarizer
from random import randint
bin=Binarizer(5)
X=[randint(0,10) for b in range(1,10)]
print(X)
print(bin.transform(X))
```

The following is the output for the preceding commands:

```
[5, 6, 1, 7, 5, 3, 3, 3, 7]
[[0 1 0 1 0 0 0 0 1]]
```

Features that are categorical often need to be encoded into integers. Consider a very simple dataset with just one categorical feature, City, with three possible values, Sydney, Perth, and Melbourne, and we decide to encode the three values as 0, 1, and 2, respectively. If this information is to be used in a linear classifier, then we write the constraint as a linear inequality with a weight parameter. The problem, however, is that this weight cannot encode for a three way choice. Suppose we have two classes, east coast and west coast, and we need our model to come up with a decision function that will reflect the fact that Perth is on the west coast and both Sydney and Melbourne are on the east coast. With a simple linear model, when the features are encoded in this way, then the decision function cannot come up with a rule that will put Sydney and Melbourne in the same class. The solution is to blow up the feature space to three features, each getting their own weights. This is called **one hot encoding**. Sciki-learn implements the `OneHotEncoder()` function to perform this task. This is an estimator that transforms each categorical feature, with *m* possible values into *m* binary features. Consider that we are using a model with data that consists of the city feature as described in the preceding example and two other features—gender, which can be either male or female, and an occupation, which can have three values—doctor, lawyer, or banker. So, for example, *a female banker from Sydney* would be represented as *[1,2,0]*. Three more samples are added for the following example:

```
from sklearn.preprocessing import OneHotEncoder
enc = OneHotEncoder()
enc.fit([[1,2,0], [1, 1, 0], [0, 2, 1], [1, 0, 2]])
print(enc.transform([1,2,0]).toarray())
```


EBSCOhost: eBook Collection (EBSCOhost) printed on 3/19/2026 5:26:39 PM UTC via TORRENS GLOBAL EDUCATION SERVICES PTY LTD. All use subject to https://www.ebsco.com/terms-of-use.

We will get the following output:

```
[[0. 1. 0. 0. 1. 1. 0. 0.]]
```

Since we have two genders, three cities, and three jobs in this dataset, the first two numbers in the transform array represent the gender, the next three represent the city, and the final three represent the occupation.

Discretization

I have already briefly mentioned the idea of thresholding in relation to decision trees, where we transform an ordinal or quantitative feature into a binary feature by finding an appropriate feature value to split on. There are a number of methods, both supervised and unsupervised, that can be used to find an appropriate split in continuous data, for example, using the statistics of central tendency (supervised), such as the mean or median or optimizing an objective function based on criteria such as information gain.

We can go further and create multiple thresholds, transforming a quantitative feature into an ordinal one. Here, we divide a continuous quantitative feature into numerous discrete ordinal values. Each of these values is referred to as a **bin**, and each bin represents an interval on the original quantitative feature. Many machine learning models require discrete values. It becomes easier and more comprehensible to create rule-based models using discrete values. Discretization also makes features more compact and may make our algorithms more efficient.

One of the most common approaches is to choose bins such that each bin has approximately the same number of instances. This is called **equal frequency discretization**, and if we apply it to just two bins, then this is the same as using the median as a threshold. This approach can be quite useful because the bin boundaries can be set up in such a way that they represent quantiles. For example, if we have 100 bins, then each bin represents a percentile.

Alternatively, we can choose the boundaries so that each bin has the same interval width. This is called **equal width discretization**. A way of working out the value of this bin's width interval is simply to divide the feature range by the number of bins. Sometimes, the features do not have an upper or lower limit, and we cannot calculate its range. In this case, integer numbers of standard deviations above and below the mean can be used. Both width and frequency discretization are unsupervised. They do not require any knowledge of the class labels to work.

EBSCOhost: eBook Collection (EBSCOhost) printed on 3/19/2026 5:26:39 PM UTC via TORRENS GLOBAL EDUCATION SERVICES PTY LTD. All use subject to https://www.ebsco.com/terms-of-use.

Let's now turn our attention to supervised discretization. There are essentially two approaches: the top-down or **divisive**, and the **agglomerative** or bottom-up approach. As the names suggest, divisive works by initially assuming that all samples belong to the same bin and then progressively splits the bins. Agglomerative methods begin with a bin for each instance and progressively merges these bins. Both methods require some stopping criteria to decide if further splits are necessary.

The process of recursively partitioning feature values through thresholding is an example of divisive discretization. To make this work, we need a scoring function that finds the best threshold for particular feature values. A common way to do this is to calculate the information gain of the split or its entropy. By determining how many positive and negative samples are covered by a particular split, we can progressively split features based on this criterion.

Simple discretization operations can be carried out by the Pandas **cut** and **qcut** methods. Consider the following example:

```
import pandas as pd
import numpy as np
print(pd.cut(np.array([1,2,3,4]), 3, retbins = True, right = False))
```

Here is the output observed:

```
([[1, 2), [2, 3), [3, 4.003), [3, 4.003)]
Categories (3, object): [[1, 2) < [2, 3) < [3, 4.003)], array([ 1.   ,  2.   ,
3.   ,  4.003]))
```

Normalization

Thresholding and discretization, both, remove the scale of a quantitative feature and, depending on the application, this may not be what we want. Alternatively, we may want to add a measure of scale to ordinal or categorical features. In an unsupervised setting, we refer to this as **normalization**. This is often used to deal with quantitative features that have been measured on a different scale. Feature values that approximate a normal distribution can be converted to z scores. This is simply a signed number of standard deviations above or below the mean. A positive z score indicates a number of standard deviations above the mean, and a negative z score indicates the number of standard deviations below the mean. For some features, it may be more convenient to use the variance rather than the standard deviation.

EBSCOhost: eBook Collection (EBSCOhost) printed on 3/19/2026 5:26:39 PM UTC via TORRENS GLOBAL EDUCATION SERVICES PTY LTD. All use subject to https://www.ebsco.com/terms-of-use.

A stricter form of normalization expresses a feature on a 0 to 1 scale. If we know a features range, we can simply use a linear scaling, that is, divide the difference between the original feature value and the lowest value with the difference between the lowest and highest value. This is expressed in the following:

$$f_n = \frac{(f - l)}{(h - l)}$$

Here, f_n is the normalized feature, f is the original feature, and l and h are the lowest and highest values, respectively. In many cases, we may have to guess the range. If we know something about a particular distribution, for example, in a normal distribution more than 99% of values are likely to fall within +3 or -3 standard deviations of the mean, then we can write a linear scaling such as the following:

$$f_n = \frac{(f - \mu)}{(6\sigma)} + \frac{1}{2}$$

Here, μ is the mean and σ is the standard deviation.

Calibration

Sometimes, we need to add scale information to an ordinal or categorical feature. This is called feature **calibration**. It is a supervised feature transformation that has a number of important applications. For example, it allows models that require scaled features, such as linear classifiers, to handle categorical and ordinal data. It also gives models the flexibility to treat features as ordinal, categorical, or quantitative. For binary classification, we can use the posterior probability of the positive class, given a features value, to calculate the scale. For many probabilistic models, such as naive Bayes, this method of calibration has the added advantage in that the model does not require any additional training once the features are calibrated. For categorical features, we can determine these probabilities by simply collecting the relative frequencies from a training set.

EBSCOhost: eBook Collection (EBSCOhost) printed on 3/19/2026 5:26:39 PM UTC via TORRENS GLOBAL EDUCATION SERVICES PTY LTD. All use subject to https://www.ebsco.com/terms-of-use.

There are cases where we might need to turn quantitative or ordinal features in to categorical features yet maintain an ordering. One of the main ways we do this is through a process of **logistic calibration**. If we assume that the feature is normally distributed with the same variance, then it turns out that we can express a likelihood ratio, the ration of positive and negative classes, given a feature value v, as follows:

$$LR(v) = \frac{(P(v \mid pos))}{(P(v \mid neg))} = exp(d'z)$$

Where d prime is the difference between the means of the two classes divided by the standard deviation:

$$d' = \frac{(\mu pos - \mu neg)}{(\sigma)}$$

Also, z is the z score:

$$\frac{(v-\mu)}{\sigma} assuming\ equal\ class\ distribution : \mu = \frac{(\mu pos + \mu neg)}{2}$$

To neutralize the effect of nonuniform class distributions, we can calculate calibrated features using the following:

$$F_c(x) = \frac{(LRF(x))}{(1 + LR(F(x)))} = exp\frac{(d'z(x))}{(1 + exp(d'z(x)))}$$

This, you may notice, is exactly the sigmoid activation function we used for logistic regression. To summarize logistic calibration, we essentially use three steps:

1. Estimate the class means for the positive and negative classes.
2. Transform the features into z scores.
3. Apply the sigmoid function to give calibrated probabilities.

Sometimes, we may skip the last step, specifically if we are using distance-based models where we expect the scale to be additive in order to calculate Euclidian distance. You may notice that our final calibrated features are multiplicative in scale.

EBSCOhost: eBook Collection (EBSCOhost) printed on 3/19/2026 5:26:39 PM UTC via TORRENS GLOBAL EDUCATION SERVICES PTY LTD. All use subject to https://www.ebsco.com/terms-of-use.

Another calibration technique, **isotonic calibration**, is used on both quantitative and ordinal features. This uses what is known as a **ROC** curve (stands for **Receiver Operator Characteristic**) similar to the coverage maps used in the discussion of logical models in *Chapter 4, Models – Learning from Information*. The difference is that with an ROC curve, we normalize the axis to *[0,1]*.

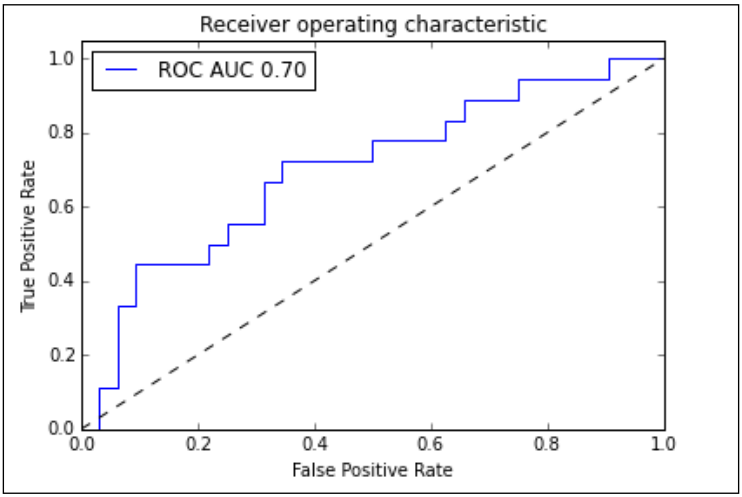
We can use the `sklearn` package to create an ROC curve:

```
import matplotlib.pyplot as plt
from sklearn import svm, datasets
from sklearn.metrics import roc_curve, auc
from sklearn.cross_validation import train_test_split
from sklearn.preprocessing import label_binarize
from sklearn.multiclass import OneVsRestClassifier

X, y = datasets.make_classification(n_samples=100,n_classes=3,n_
features=5, n_informative=3, n_redundant=0,random_state=42)
# Binarize the output
y = label_binarize(y, classes=[0, 1, 2])
n_classes = y.shape[1]
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=.5)
classifier = OneVsRestClassifier(svm.SVC(kernel='linear',
probability=True, ))
y_score = classifier.fit(X_train, y_train).decision_function(X_test)
fpr, tpr, _ = roc_curve(y_test[:,0], y_score[:,0])
roc_auc = auc(fpr, tpr)
plt.figure()
plt.plot(fpr, tpr, label='ROC AUC %0.2f' % roc_auc)
plt.plot([0, 1], [0, 1], 'k--')
plt.xlim([0.0, 1.0])
plt.ylim([0.0, 1.05])
plt.xlabel('False Positive Rate')
plt.ylabel('True Positive Rate')
plt.title('Receiver operating characteristic')
plt.legend(loc="best")
plt.show()
```


EBSCOhost: eBook Collection (EBSCOhost) printed on 3/19/2026 5:26:39 PM UTC via TORRENS GLOBAL EDUCATION SERVICES PTY LTD. All use subject to https://www.ebsco.com/terms-of-use.

Here is the output observed:



The ROC curve maps the true positive rates against the false positive rate for different threshold values. In the preceding diagram, this is represented by the dotted line. Once we have constructed the ROC curve, we calculate the number of positives, m_i, and the total number of instances, n_i, in each segment of the convex hull. The following formula is then used to calculate the calibrated feature values:

$$v_c = \frac{(m_i + 1)}{(m_i + 1 + c(n_i - m_i + 1))}$$

In this formula, c is the prior odds, that is, the ratio of the probability of the positive class over the probability of the negative class.

So far in our discussion on feature transformations, we assumed that we know all the values for every feature. In the real world, this is often not the case. If we are working with probabilistic models, we can estimate the value of a missing feature by taking a weighted average over all features values. An important consideration is that the existence of missing feature values may be correlated with the target variable. For example, data in an individual's medical history is a reflection of the types of testing that are performed, and this in turn is related to an assessment on risk factors for certain diseases.

EBSCOhost: eBook Collection (EBSCOhost) printed on 3/19/2026 5:26:39 PM UTC via TORRENS GLOBAL EDUCATION SERVICES PTY LTD. All use subject to https://www.ebsco.com/terms-of-use.

If we are using a tree model, we can randomly choose a missing value, allowing the model to split on it. This, however, will not work for linear models. In this case, we need to fill in the missing values through a process of **imputation**. For classification, we can simply use the statistics of the mean, median, and mode over the observed features to impute the missing values. If we want to take feature correlation into account, we can construct a predictive model for each incomplete feature to predict missing values.

Since scikit-learn estimators always assume that all values in an array are numeric, missing values, either encoded as blanks, NaN, or other placeholders, will generate errors. Also, since we may not want to discard entire rows or columns, as these may contain valuable information, we need to use an imputation strategy to complete the dataset. In the following code snippet, we will use the `Imputer` class:

```
from sklearn.preprocessing import Binarizer, Imputer, OneHotEncoder
imp = Imputer(missing_values='NaN', strategy='mean', axis=0)
print(imp.fit_transform([[1, 3], [4, np.nan], [5, 6]]))
```

Here is the output:

```
[[ 1.   3. ]
 [ 4.   4.5]
 [ 5.   6. ]]
```

Many machine learning algorithms require that features are **standardized**. This means that they will work best when the individual features look more or less like normally distributed data with near-zero mean and unit variance. The easiest way to do this is by subtracting the mean value from each feature and scaling it by dividing by the standard deviation. This can be achieved by the `scale()` function or the `standardScaler()` function in the `sklearn.preprocessing()` function. Although these functions will accept sparse data, they probably should not be used in such situations because centering sparse data would likely destroy its structure. It is recommended to use the `MacAbsScaler()` or `maxabs_scale()` function in these cases. The former scales and translates each feature individually by its maximum absolute value. The latter scales each feature individually to a range of *[-1,1]*. Another specific case is when we have outliers in the data. In these cases using the `robust_scale()` or `RobustScaler()` function is recommended.

Often, we may want to add complexity to a model by adding polynomial terms. This can be done using the `PolynomialFeatures()` function:

```
from sklearn.preprocessing import PolynomialFeatures
X=np.arange(9).reshape(3,3)
```


EBSCOhost: eBook Collection (EBSCOhost) printed on 3/19/2026 5:26:39 PM UTC via TORRENS GLOBAL EDUCATION SERVICES PTY LTD. All use subject to https://www.ebsco.com/terms-of-use.

```
poly=PolynomialFeatures(degree=2)
print(X)
print(poly.fit_transform(X))
```

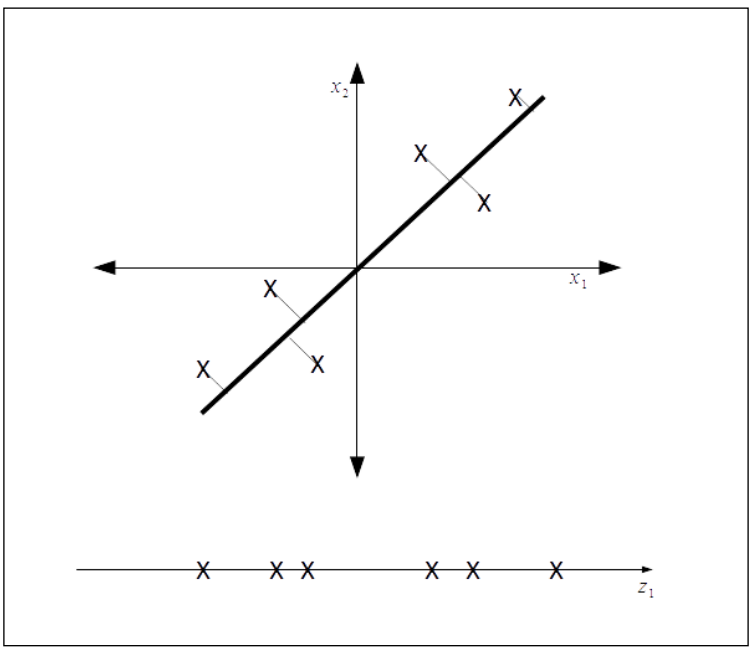
We will observe the following output:

$$\begin{bmatrix}\begin{bmatrix}0 & 1 & 2\end{bmatrix} \\ \begin{bmatrix}3 & 4 & 5\end{bmatrix} \\ \begin{bmatrix}6 & 7 & 8\end{bmatrix}\end{bmatrix}$$

$$\begin{bmatrix}\begin{bmatrix}1 & 0 & 1 & 2 & 0 & 0 & 0 & 1 & 2 & 4\end{bmatrix} \\ \begin{bmatrix}1 & 3 & 4 & 5 & 9 & 12 & 15 & 16 & 20 & 25\end{bmatrix} \\ \begin{bmatrix}1 & 6 & 7 & 8 & 36 & 42 & 48 & 49 & 56 & 64\end{bmatrix}\end{bmatrix}$$

Principle component analysis

Principle Component Analysis (**PCA**) is the most common form of dimensionality reduction that we can apply to features. Consider the example of a dataset consisting of two features and we would like to convert this two-dimensional data into one dimension. A natural approach would be to draw a line of the closest fit and project each data point onto this line, as shown in the following graph:



EBSCOhost: eBook Collection (EBSCOhost) printed on 3/19/2026 5:26:39 PM UTC via TORRENS GLOBAL EDUCATION SERVICES PTY LTD. All use subject to https://www.ebsco.com/terms-of-use.

PCA attempts to find a surface to project the data by minimizing the distance between the data points and the line we are attempting to project this data to. For the more general case where we have *n* dimensions and we want to reduce this space to k-dimensions, we find *k* vectors *u(1),u(2), ..., u(k)* onto which to project the data so as to minimize the projection error. That is we are trying to find a k-dimensional surface to project the data.

This looks superficially like linear regression however it is different in several important ways. With linear regression we are trying to predict the value of some output variable given an input variable. In PCA we are not trying to predict an output variable, rather we are trying to find a subspace onto which to project our input data. The error distances, as represented in the preceding graph, is not the vertical distance between a point and the line, as is the case for linear regression, but rather the closest orthogonal distance between the point and the line. Thus, the error lines are at an angle to the axis and form a right angle with our projection line.

An important point is that in most cases, PCA requires that the features are scaled and are mean normalized, that is, the features have zero mean and have a comparable range of values. We can calculate the mean using the following formula:

$$\mu_j = \frac{1}{m}\sum_{i=1}^{m} x_j^{(i)}$$

The sum is calculated by replacing the following:

$$x_j^{(i)} with\ x_j - \mu_j$$

If the features have scales that are significantly different, we can rescale using the following:

$$\frac{\left(x_j - \mu_j\right)}{\sigma_j}$$

These functions are available in the `sklearn.preprocessing` module.

EBSCOhost: eBook Collection (EBSCOhost) printed on 3/19/2026 5:26:39 PM UTC via TORRENS GLOBAL EDUCATION SERVICES PTY LTD. All use subject to https://www.ebsco.com/terms-of-use.

The mathematical process of calculating both the lower dimensional vectors and the points on these vectors where we project our original data involve first calculating the covariance matrix and then calculating the eigenvectors of this matrix. To calculate these values from first principles is quite a complicated process. Fortunately, the `sklearn` package has a library for doing just this:

```
from sklearn.decomposition import PCA
import numpy as np
X = np.array([[-1, -1], [-2, -1], [-3, -2], [1, 1], [2, 1], [3, 2]])
pca = PCA(n_components=1)
pca.fit(X)
print(pca.transform(X))
```

We will get the following output:

$$\begin{bmatrix} [-1.3834058] \\ [-2.22189802] \\ [-3.60530382] \\ [1.3834058\ \ ] \\ [2.22189802\ ] \\ [3.60530382\ ] \end{bmatrix}$$

Summary

There are a rich variety of ways in which we can both transform and construct new features to make our models work more efficiently and give more accurate results. In general, there are no hard and fast rules for deciding which of the methods to use for a particular model. Much depends on the feature types (quantitative, ordinal, or categorical) that you are working with. A good first approach is to normalize and scale the features, and if the model requires it, transform the feature to an appropriate type, as we do through discretization. If the model performs poorly, it may be necessary to apply further preprocessing such as PCA. In the next chapter, we will look at ways in which we can combine different types of models, through the use of ensembles, to improve performance and provide greater predictive power.

EBSCOhost: eBook Collection (EBSCOhost) printed on 3/19/2026 5:26:39 PM UTC via TORRENS GLOBAL EDUCATION SERVICES PTY LTD. All use subject to https://www.ebsco.com/terms-of-use.

EBSCOhost: eBook Collection (EBSCOhost) printed on 3/19/2026 5:26:39 PM UTC via TORRENS GLOBAL EDUCATION SERVICES PTY LTD. All use subject to https://www.ebsco.com/terms-of-use.